**Performance Testing**

| | |
|---|---|
| **Date** | 7 July 2026 |
| **Team ID** | xxxxxx |
| **Project Name** | **Credit Card Approval Prediction** |
| **Maximum Marks** | 5 Marks |

**Performance Testing**

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

**Step 1: Testing Overview**

| Field | Details |
|---|---|
| **Testing Tool Used** | Postman, Flask Local Server |
| **Type of Testing** | Load Tes5ng, Func5onal Tes5ng |
| **Target Module / API** | Credit Card Approval Predic5on API |
| **Test Environment** | Local Machine (Windows/macOS, Flask Server) |
| **Test Date** | 7 July 2026 |

**Step 2: Test Scenarios**

| S.No | Test Scenario / Description | No. of Virtual Users | Duration (sec) | Expected Outcome |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| 1 | Single user predic5on request | 1 | 30 | Predic5on returned successfully |
| 2 | Mul5ple predic5on requests | 10 | 60 | All requests processed correctly |
| 3 | Invalid input tes5ng | 5 | 30 | Appropriate error message displayed |
| 4 | Con5nuous predic5on requests | 20 | 120 | Stable performance without crashes |



**Step 3: Performance Test Results**

| S.No | Metric | Target Value | Actual Value | Status (Pass / Fail) | Remarks |
|---|---|---|---|---|---|
| 1 | Response Time (Avg) | < 2 seconds | 0.82 sec | Pass | Fast response |
| 2 | Response Time (Max) | < 5 seconds | 2.10 sec | Pass | Within acceptable limit |
| 3 | Throughput (Req/sec) | 15 Req/sec | 18 Req/sec | Pass | Good throughput |
| 4 | Error Rate | < 1% | 0% | Pass | No request failures |
| 5 | CPU Utilization | < 80% | 48% | Pass | Stable CPU usage |
| 6 | Memory Utilization | < 80% | 55% | Pass | Memory usage within limit |

**Step 4: Observations & Analysis**

**Key Findings**

- The prediction system responded quickly under normal and moderate workloads.
- All prediction requests were processed successfully with no failures.
- Response time remained below the target threshold during testing.

**Bottlenecks Identified**

- Minor increase in response time during continuous requests due to model loading.
- No major performance bottlenecks were observed.

**Optimization Steps Taken**

- Optimized data preprocessing functions.
- Reduced redundant computations.
- Used efficient Random Forest model parameters.
- Improved Flask application response handling.

**Step 5: Screenshots / Evidence**

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

[Insert Screenshot 1 here]

[Insert Screenshot 2 here]